

Estrutura runtime e arquitetura de aplicações *agentic* em escala

Guia de preparação — Especialista de domínio / Principal em IA aplicada a seguros · Itaú · Cloud AWS

ITEM DE RESPONSABILIDADE DA VAGA

"Estrutura runtime e arquitetura de aplicações *agentic* em escala."

1. Por que isso é perguntado

Porque quase todo mundo já colocou *um* agente em produção — e quase ninguém colocou *cem*. O entrevistador quer saber se você sabe a diferença entre um agente e uma **plataforma de agentes**: quem é dono do loop, onde mora o estado, quem paga o token, quem consegue auditar a decisão seis meses depois, e o que acontece quando o agente chama uma tool que move dinheiro duas vezes.

O que está sendo testado, em nível Principal:

- **Separação plano de controle / plano de dados.** Se você descreve deploy, versionamento, policy e catálogo de tools misturados com o caminho da requisição, você não desenhou uma plataforma — desenhou uma aplicação grande.
- **Modelo de execução.** Sessão longa e *stateful*, com passos não-determinísticos e efeitos colaterais externos, não é request/response. Se você trata como request/response, o sistema quebra em timeout, custo e idempotência.
- **Fronteiras de isolamento.** Multi-LOB (vida, auto, residencial, prestamista) sob uma seguradora regulada exige responder "onde termina o tenant?" em quatro camadas: compute, credencial, dado e orçamento.
- **Bounded autonomy.** Quanto de autonomia você concede, e qual é o mecanismo *determinístico* que impede o agente de exceder — não "o prompt diz para não fazer".

2. Fundamentos

O runtime agentic não é um runtime de microserviço

Um microserviço típico é sem estado, curto, determinístico e com fan-out conhecido em tempo de design. Um agente inverte as quatro propriedades:

PROPRIEDADE	MICROSERVIÇO	APLICAÇÃO AGENTIC	CONSEQUÊNCIA ARQUITETURAL
Estado	Stateless; estado no banco	Contexto de sessão vivo entre turnos (memória curta) + memória longa entre sessões	Exige <i>session affinity</i> e um serviço de memória separado do compute
Duração	ms a segundos	Minutos a horas (regulação de sinistro documental, pesquisa em múltiplas bases)	Invocação assíncrona, streaming, e checkpoint durável
Determinismo	Mesma entrada → mesma saída	Estocástico; o caminho de execução muda a cada chamada	Teste por avaliação comportamental, não por asserção exata; observabilidade por <i>trace</i> , não por log
Fan-out	Conhecido em design time	Decidido pelo modelo em runtime (N chamadas de LLM × M tools)	Custo e latência viram variáveis aleatórias → orçamento explícito de passos e tokens

Os cinco eixos que definem “o runtime”

- Unidade de isolamento.** No `Bedrock AgentCore Runtime` a unidade é a sessão: cada sessão recebe uma microVM dedicada com CPU, memória e filesystem isolados; ao término, a microVM é destruída e a memória higienizada. Duração máxima de 8 h de runtime total; término por inatividade em 15 min. Isso resolve contaminação cross-sessão “de graça” — mas **não** resolve mapeamento usuário↔sessão: a documentação é explícita de que o serviço não impõe esse vínculo, e o seu backend precisa mantê-lo (inclusive limite de sessões por usuário).
- Afinidade e roteamento.** O `runtimeSessionId` é o que roteia a requisição para a mesma microVM. Perder o session ID não dá erro — dá uma microVM nova, contexto perdido e *cold start*. Esse é o bug silencioso mais caro do modelo.

3. **Contrato de invocação.** HTTP request/response, streaming (SSE), WebSocket bidirecional, MCP (agente↔tool) e A2A (agente↔agente). A escolha do protocolo é decisão de arquitetura, não de framework: A2A abre a porta para descoberta entre agentes de times diferentes, e com isso para uma superfície de confiança que precisa ser governada.
4. **Identidade em dois sentidos.** *Inbound:* IAM SigV4 (serviço→serviço) ou OAuth 2.0/JWT (usuário final via IdP) — um runtime suporta um método por vez, então ambientes com os dois modos exigem versões/endpoints separados. *Outbound:* AgentCore Identity gerencia OAuth e API keys em modo **delegado ao usuário** (agente age como o cliente) ou **autônomo** (agente age como o serviço). Em seguros, essa distinção é a diferença entre “consultei a apólice do titular autenticado” e “varri a base”.
5. **Versionamento e promoção.** Versões são imutáveis e capturam configuração completa (imagem, protocolo, rede). *Endpoints* apontam para versões e são o ponto estável de invocação; o endpoint `DEFAULT` segue a última versão automaticamente — o que é conveniente em dev e **perigoso em produção**. Endpoints nomeados por ambiente com promoção explícita são o padrão correto.

O que não é óbvio

- **A microVM é a fronteira, e ela vaza credencial para dentro.** Qualquer código rodando na microVM alcança as credenciais da execution role pelo endpoint de metadados (MMDS), do mesmo jeito que o IMDS no EC2. Ou seja: *prompt injection que consegue executar código herda a execution role*. Consequência direta: a execution role tem que ser escopada como se fosse hostil, e o container deve rodar como usuário não-root.
- **Autorização é hierárquica.** Para `InvokeAgentRuntime`, a AWS avalia políticas *tanto* no agent runtime quanto no agent endpoint — as duas precisam permitir. Cross-account exige resource policy nos dois recursos.
- **Delegação de usuário é uma ação separada.** `InvokeAgentRuntimeForUser` e o header `X-Amzn-Bedrock-AgentCore-Runtime-User-Id` precisam derivar do principal autenticado, nunca de valor arbitrário do cliente; onde não é necessário, a recomendação é *negar explicitamente* a ação.
- **Estado de sessão é efêmero por design.** A doc diz para não usá-lo como durabilidade — durabilidade é AgentCore Memory (curta e longa) ou seu próprio store. Um agente que “lembra” só porque a microVM ainda está viva vai falhar no primeiro *idle timeout* de 15 minutos.
- **Autonomia limitada precisa de mecanismo determinístico.** O AgentCore `Policy` (Cedar) intercepta cada chamada de tool no Gateway antes da execução. Isso é qualitativamente diferente de instruir o modelo: é um ponto de controle fora do LLM. É o argumento que você usa quando o time de risco perguntar “e se o modelo alucinar a intenção de pagar?”.

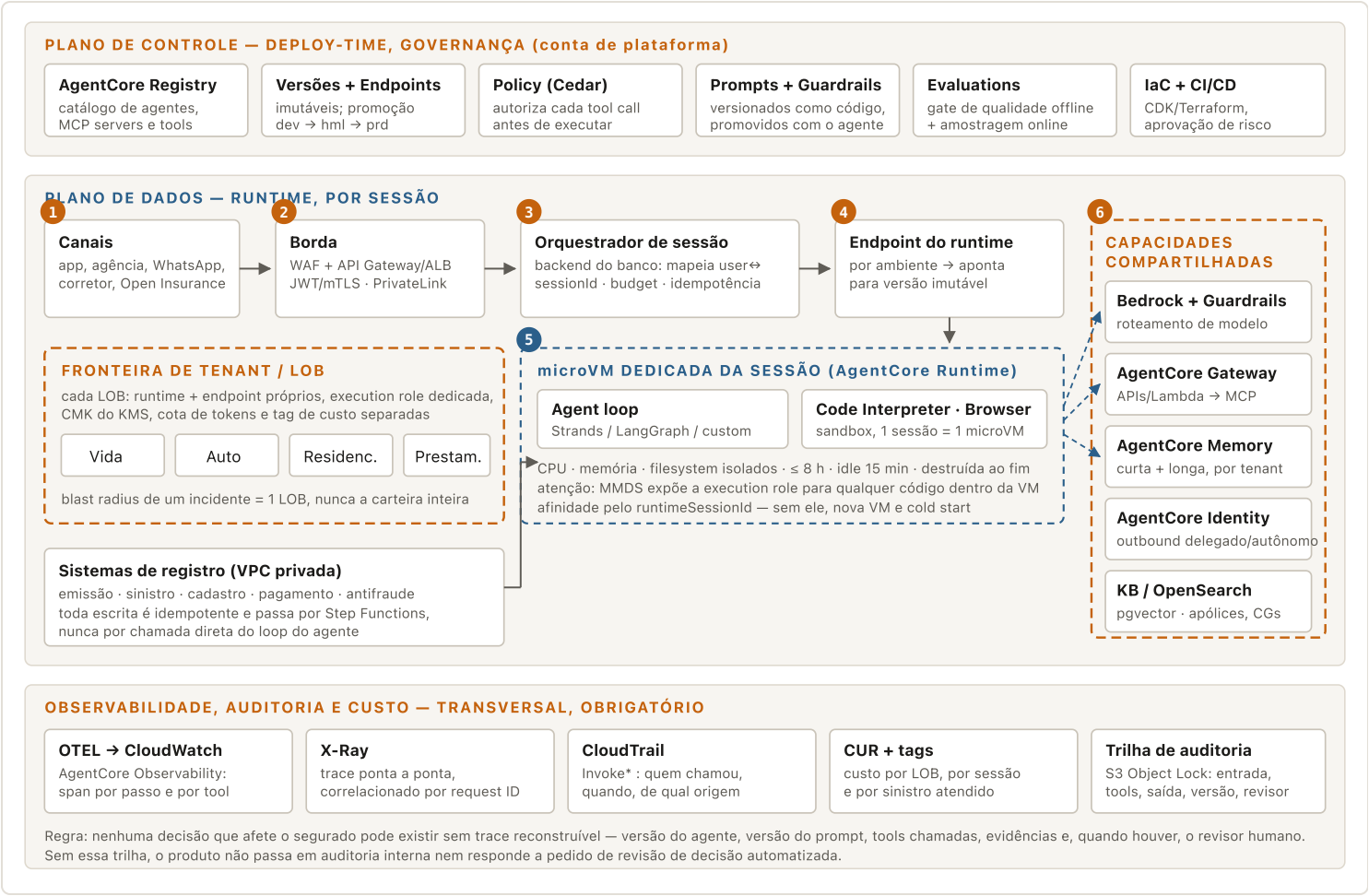
Padrões de orquestração e onde cada um quebra

- **Agente único com tools.** Mais barato e mais fácil de auditar. Quebra quando o número de tools cresce: acima de algumas dezenas, a seleção de tool degrada e o custo do prompt de sistema domina. Correção: catálogo por domínio + roteamento antes do agente.
- **Supervisor / arbiter com sub-agentes.** Boa fronteira organizacional (um sub-agente por LOB, com seu próprio dono). Custo: cada handoff é uma chamada de LLM adicional e um ponto de perda de contexto. Só compensa quando os sub-agentes têm ferramentas e permissões genuinamente distintas.
- **Workflow determinístico com passos agentic.** A máquina de estados é sua (Step Functions); o agente resolve apenas os nós ambíguos. É o padrão que mais se aproxima do que um regulador aceita, porque a trilha de auditoria é a execução do workflow, não a cadeia de pensamento.
- **Swarm / peer-to-peer.** Quase nunca justificável em seguros hoje: latência e custo não-limitados, e responsabilidade difusa. Ter uma opinião sobre isso — e saber dizer “não” — é o que diferencia Principal de sênior.

A regra prática que vale enunciar em entrevista: **determinismo por padrão, autonomia por exceção justificada.** Cada grau de autonomia adicional precisa comprar algo que o workflow não entrega.

3. Arquitetura de referência

Plataforma multi-LOB de agentes para seguros: um plano de controle centralizado (governança, catálogo, versões, policy, avaliação) operado pela plataforma; um plano de dados por LOB, com fronteira de tenant em compute, credencial, chave e orçamento; capacidades compartilhadas (modelo, memória, tools, conhecimento) atrás de contratos versionados; observabilidade e custo transversais.



Caminho da requisição: (1) canal → (2) borda com WAF e autenticação → (3) orquestrador de sessão do banco, que é o dono do vínculo usuário ↔ `sessionId`, do orçamento de passos e da idempotência → (4) endpoint versionado do LOB → (5) microVM dedicada da sessão executa o loop → (6) capacidades compartilhadas atendem modelo, tools, memória, identidade e conhecimento. Escritas em sistema de registro nunca saem direto do loop: passam por workflow idempotente.

4. Mapa de serviços AWS

PROBLEMA	SERVIÇO PRIMÁRIO	ALTERNATIVA	QUANDO ESCOLHER CADA UM	PEGADINHA OPERACIONAL
Hospedar o loop do agente com isolamento por sessão	Bedrock AgentCore Runtime	ECS Fargate / EKS com sticky session própria	AgentCore quando você quer isolamento por sessão, identidade e observabilidade sem construir; ECS/EKS quando já	Perder o <code>runtimeSessionId</code> não gera erro — gera microVM nova e cold start. Endpoint <code>DEFAULT</code> segue a última versão

PROBLEMA	SERVIÇO PRIMÁRIO	ALTERNATIVA	QUANDO ESCOLHER CADA UM	PEGADINHA OPERACIONAL
			existe malha de serviço, sidecars obrigatórios ou requisito de rede que o serviço não cobre	automaticamente: nunca aponte produção para ele
Loop gerenciado sem escrever orquestração	AgentCore Harness	Strands Agents / LangGraph no Runtime	Harness para agentes internos e prototipagem rápida; framework explícito quando você precisa controlar política de retry, corte de contexto e handoff	Loop gerenciado = menos pontos de instrumentação próprios; valide se o nível de trace atende à sua exigência de auditoria antes de padronizar
Expor APIs internas como tools	AgentCore Gateway (MCP)	API Gateway + Lambda chamado direto pelo agente	Gateway quando as mesmas tools servem vários times — é onde o Policy intercepta; chamada direta só para tool exclusiva de um agente	Tool exposta é superfície de ataque nova: um <code>GET</code> de apólice sem filtro de titularidade vira exfiltração via prompt injection
Autorização determinística de tool call	AgentCore Policy (Cedar)	Verificação no código da tool / Lambda authorizer	Cedar quando a regra é transversal e precisa ser auditável fora do código; validação local para invariantes de negócio específicos	Policy não substitui autorização no sistema de registro — defesa em profundidade, não em vez de
Memória curta e longa do agente	AgentCore Memory	DynamoDB / Aurora + resumo próprio	Memory quando quer estratégias gerenciadas de extração e	Memória longa é dado pessoal sob LGPD: precisa de política de retenção, escopo por

PROBLEMA	SERVIÇO PRIMÁRIO	ALTERNATIVA	QUANDO ESCOLHER CADA UM	PEGADINHA OPERACIONAL
			compartilhamento entre agentes; store próprio quando retenção, schema e exclusão precisam ser 100% seus	titular e caminho de exclusão. Estado de sessão é efêmero — não confie nele para durabilidade
Identidade de entrada	IAM SigV4	OAuth 2.0 / JWT (Cognito, Okta, Entra ID)	SigV4 para chamadas serviço→serviço dentro da AWS; JWT quando o usuário final autentica no IdP e você quer o claim dele no runtime	Um runtime suporta um método por vez — misturar exige versões/endpoints separados. Configure <i>todos</i> os campos do authorizer (audiences, clients, scopes, claims)
Credenciais de saída para sistemas terceiros	AgentCore Identity	Secrets Manager + código próprio	Identity para OAuth delegado ao usuário; Secrets Manager para credencial de serviço de sistema legado sem OAuth	Escolher modo autônomo por conveniência apaga a rastreabilidade de “em nome de quem” — em seguros isso derruba a auditoria
Escrita transacional e passos de longa duração	Step Functions	SQS + Lambda / EventBridge	Step Functions quando existem compensação, aprovação humana e retomada; fila quando é fire-and-forget idempotente	Deixar o agente escrever direto no sistema de registro remove o ponto de compensação. Toda tool de escrita precisa de chave de idempotência gerada fora do LLM
Human-in-the-loop	Step Functions (task token) + SQS	Fila de trabalho no sistema de sinistro/subscrição	Task token quando a espera é parte do fluxo do agente; fila de trabalho quando	HITL implementado como “sessão parada esperando” colide com o timeout de

PROBLEMA	SERVIÇO PRIMÁRIO	ALTERNATIVA	QUANDO ESCOLHER CADA UM	PEGADINHA OPERACIONAL
			o revisor já opera em outra ferramenta	inatividade e com o teto de duração — a espera tem que ser durável fora da microVM
Conhecimento (condições gerais, manuais de regulação)	Bedrock Knowledge Bases	OpenSearch Serverless / Aurora pgvector	KB para caminho gerenciado; índice próprio quando precisa de filtro multi-tenant fino, reranking próprio ou custo previsível em volume alto	Filtro de tenant no índice é obrigatório: recuperar CG de outro produto gera resposta errada com aparência correta
Cache de respostas e de contexto	ElastiCache (cache semântico)	Prompt caching no Bedrock	Cache semântico para FAQ e dúvidas repetidas de canal; prompt caching para prefixo de sistema longo e estável	Cache sem chave de tenant e de perfil vaza resposta entre segurados — chave sempre composta por tenant + escopo de dado
Rede privada	VPC + PrivateLink	NAT Gateway com egress controlado	Interface endpoints para data plane (<code>bedrock-agentcore</code>) e control plane (<code>bedrock-agentcore-control</code>); NAT quando há dependência externa legítima	Subnet pública não dá internet ao Runtime: use subnet privada com rota para NAT. Em modo container, faltam com frequência os endpoints de ECR, S3 gateway e CloudWatch Logs
Observabilidade do raciocínio	AgentCore Observability (OTEL) + CloudWatch	X-Ray + coletor OTEL próprio	Nativo para span por passo do agente e por tool; coletor próprio	Métrica agregada esconde a cauda: p50 de agente é irrelevante, o que dói

PROBLEMA	SERVIÇO PRIMÁRIO	ALTERNATIVA	QUANDO ESCOLHER CADA UM	PEGADINHA OPERACIONAL
			quando precisa exportar para stack corporativa existente	é p95 de sessão com 9 passos
Avaliação contínua	AgentCore Evaluations	Bedrock Evaluations / harness próprio	Evaluations sobre sessões e traces reais quando o objeto avaliado é a trajetória; Bedrock Evaluations quando o objeto é a resposta do modelo	Avaliar só a resposta final esconde caminhos caros e tool calls indevidas que deram certo por sorte
Custo por unidade de negócio	Cost & Usage Report + tags + Budgets	Contas separadas por LOB	Tags quando a plataforma é compartilhada; conta separada quando o LOB exige limite rígido e isolamento de faturamento	Custo de agente precisa ser medido por <i>tarefa concluída</i> (por sinistro triado), não por 1.000 tokens — senão a conversa com produto não fecha
Segregação de contas	Organizations: OU de IA (SRA for AI)	Conta única com boundaries de IAM	OU dedicada com conta de aplicação e conta de IA separadas, como recomenda a SRA; conta única só em ambiente de experimentação	SCP escrita depois do primeiro workload em produção sempre quebra alguém — escreva antes

5. Trade-offs estruturantes

1. Runtime gerenciado (AgentCore) × runtime próprio (ECS/EKS)

Gerenciado. Isolamento por sessão, identidade inbound/outbound, observabilidade OTEL e versionamento prontos. Menos código de plataforma para manter, menos superfície para errar.

Próprio. Controle total de rede, sidecars, malha de serviço e política de scheduling. Reaproveita a esteira de container que o banco já opera há anos.

Critério de decisão: a restrição que decide não é técnica, é organizacional — quem vai operar isso às 3h da manhã, e o que o time de infra já sabe operar. Tecnicamente, o divisor é: você precisa de isolamento forte por sessão de usuário final? Se sim, construir isso em ECS custa caro e erra fácil.

Recomendação: gerenciado como caminho padrão (*paved road*) para agentes que atendem cliente final; runtime próprio como exceção documentada para casos com requisito de rede ou de sidecar que o serviço não cobre. Ter dois caminhos é aceitável; ter dois caminhos *sem* declarar qual é o padrão é o erro.

2. Autonomia do agente × workflow determinístico

Autonomia. O agente decide o plano. Cobre cauda longa de casos, adapta-se a documento fora do padrão, reduz o custo de manter regras.

Workflow. Máquina de estados explícita; o agente só resolve nós ambíguos. Latência e custo previsíveis, trilha de auditoria trivial, teste convencional.

Critério de decisão: reversibilidade e materialidade da ação. Ação reversível e de baixo valor (resumir apólice, classificar documento) tolera autonomia. Ação irreversível ou com efeito financeiro (aprovar indenização, alterar cobertura, cancelar apólice) não tolera — não por medo do modelo, mas porque a instituição precisa apontar o passo que autorizou.

Recomendação: workflow determinístico como esqueleto, com nós agentic. Autonomia cresce por faixa de valor, com limite duro e revisão humana acima do limite. É a leitura que se sustenta tanto tecnicamente quanto diante do regulador.

3. Sessão longa e viva x checkpoint durável

Sessão viva. Contexto quente, latência menor por turno, código simples — o estado está na memória da microVM.

Checkpoint. Cada passo persiste estado e é retomável de outro processo. Sobrevive a timeout de inatividade, deploy e falha de infraestrutura.

Critério de decisão: qual é a janela real entre interações? Conversa de atendimento tem segundos entre turnos; regulação de sinistro documental tem horas ou dias, porque espera perícia, documento faltante ou revisor. O teto de 8 h e o corte por 15 min de inatividade tornam a sessão viva inviável para o segundo caso.

Recomendação: sessão viva para interação conversacional; checkpoint durável (Step Functions + memória persistida) para qualquer processo que atravesse a espera de um humano ou de um sistema externo. Nunca desenhe HITL como “a sessão fica esperando”.

4. Multi-tenant compartilhado x silo por LOB

Compartilhado. Um runtime, tenant resolvido por claim e por filtro. Custo unitário menor, upgrade em um lugar, capacidade agregada absorve pico.

Silo. Runtime, execution role, CMK e cota por LOB. Blast radius pequeno, atribuição de custo trivial, cadência de deploy independente.

Critério de decisão: a pergunta correta não é “compartilhado ou silo?”, é “em qual camada fica a fronteira?”. Isolamento de *compute* por sessão já vem da microVM. O que precisa de decisão explícita é credencial, chave, índice de conhecimento, memória e orçamento — e essas quatro podem ser siloadas mesmo com plataforma compartilhada.

Recomendação: plano de controle compartilhado, execution role e CMK por LOB, índice e memória com filtro obrigatório de tenant, e cota de token por LOB. Silo completo de runtime só para o LOB com exigência regulatória ou contratual específica — e nesse caso é decisão de conformidade, não de arquitetura.

5. Multi-agente x agente único com muitas tools

Multi-agente. Fronteiras claras de responsabilidade e de time; contexto de cada agente permanece pequeno; permissões naturalmente segregadas.

Agente único. Sem overhead de handoff, um trace só, um lugar para depurar, custo menor por tarefa.

Critério de decisão: cada handoff custa uma inferência a mais, uma perda parcial de contexto e um modo de falha distribuída novo. Só vale quando os sub-agentes precisam de *permissões diferentes* ou têm *donos diferentes*. "Fica mais organizado" não é critério.

Recomendação: começar com agente único e catálogo de tools curado por domínio. Fragmentar quando a seleção de tool degradar de forma mensurável na avaliação, ou quando a segregação de permissão exigir. Multi-agente com padrão supervisor; *swarm* só com justificativa que sobreviva a uma revisão de custo.

6. Streaming ao usuário x validação antes de exibir

Streaming. Percepção de latência muito melhor no canal digital; token aparece enquanto o agente ainda raciocina.

Validar primeiro. Guardrail e verificação de fatos rodam sobre a resposta completa antes de qualquer byte chegar ao segurado.

Critério de decisão: o que acontece se o texto exibido estiver errado? Explicação genérica de cobertura tolera correção posterior. Afirmação sobre valor de indenização, carência ou exclusão não tolera — texto exibido a segurado é conduta perante o consumidor e já foi visto.

Recomendação: streaming por padrão no canal digital, com *gate* não-streaming para blocos que contenham valor, prazo, exclusão ou decisão. Na prática: streaming da parte explicativa, bloco final validado. Nunca "stream e corrige depois".

6. Aterrissagem em seguros no Itaú

Caso 1 — Triagem e regulação assistida de sinistro residencial e auto

SESSÃO LONGA

MULTIMODAL

HITL POR FAIXA DE VALOR

TRILHA DE AUDITORIA

LGPD ART. 20

ESCALA ESTIMADA

O fluxo: o segurado abre o aviso pelo app com fotos e nota fiscal. O agente extrai dados do documento, consulta apólice e cobertura vigente, checa consistência entre relato, imagens e histórico, aciona antifraude e produz um parecer com evidências. Abaixo de um limite de valor e com confiança alta, segue para pagamento automático; acima, vai para fila do regulador humano com o parecer pré-montado.

Impacto: o ganho relevante não é “o agente decide”, é encurtar o tempo de ciclo e entregar ao regulador humano um dossiê pronto. O modelo de negócio melhora por *throughput* de regulador, não por substituição.

Restrição regulatória: decisão automatizada que afete o segurado precisa ser explicável e revisável — a LGPD assegura ao titular o direito de solicitar revisão de decisões tomadas unicamente com base em tratamento automatizado. A trilha precisa guardar entrada, documentos, tools chamadas, versão do agente e do prompt, e o revisor quando houver. Regras setoriais de segurança cibernética e de contratação de nuvem para supervisionados da SUSEP aplicam-se ao ambiente (*verificar a norma vigente e suas atualizações na fonte primária antes de citar número em entrevista*).

Como o tema do dia muda a decisão: este caso é o exemplo canônico de por que a sessão viva não serve. Falta documento, entra perícia, o revisor volta no dia seguinte — a espera atravessa o teto de 8 h e o corte por inatividade. O desenho correto é Step Functions como dono do caso, com task token para a espera humana, e o agente invocado por etapa, recuperando contexto de memória persistida. Cada tool de escrita (registrar parecer, autorizar pagamento) carrega chave de idempotência gerada pelo workflow, nunca pelo modelo.

ESTIMATIVA PARA DIMENSIONAR, NÃO NÚMERO OFICIAL

uma carteira massificada de banco grande pode gerar dezenas de milhares de avisos por mês; com pico concentrado em evento climático, a carga instantânea pode ser 5–10× a média diária. Isso decide capacidade reservada, cota por LOB e política de degradação.

Caso 2 — Atendimento e venda assistida no canal digital (app e WhatsApp)

BAIXA LATÊNCIA

STREAMING

CACHE SEMÂNTICO

CONDUTA E DEVER DE INFORMAÇÃO

RASTREABILIDADE DA OFERTA

O fluxo: o cliente pergunta sobre cobertura, simula um seguro viagem ou pede segunda via. O agente responde com base em condições gerais indexadas, consulta a apólice do titular autenticado, e — quando cabe — apresenta oferta. Contratação efetiva sai do loop do agente e entra no fluxo transacional do produto.

Impacto: deflexão de atendimento humano e aumento de conversão em canal digital. O custo por conversa importa mais do que o custo por token: uma conversa de 12 turnos com 3 tool calls por turno pode custar uma ordem de grandeza mais do que a estimativa ingênua.

Restrição regulatória: tudo que o agente afirma sobre cobertura, carência e exclusão é dever de informação. A oferta apresentada precisa ser reconstruível: qual versão de condição geral foi recuperada, qual texto foi exibido, em que momento. Dado pessoal em memória longa exige base legal, retenção definida e caminho de exclusão.

Como o tema do dia muda a decisão: aqui a sessão viva é o desenho certo — afinidade por `runtimeSessionId`, contexto quente, streaming. Mas três decisões de runtime dominam a experiência: (i) manter o session ID no cliente e no orquestrador, porque perdê-lo troca cold start por “o bot esqueceu tudo”; (ii) cache semântico com chave composta por tenant, produto e escopo de titularidade, senão a resposta de um segurado vaza para outro; (iii) *gate* não-streaming no bloco que cita valor, prazo ou exclusão.

Caso 3 — Copiloto de subscrição e retenção para operação interna

MULTI-AGENTE SUPERVISOR

TOOLS DE ALTO PRIVILÉGIO

POLICY DETERMINÍSTICA

SEGREGAÇÃO DE FUNÇÃO

NÃO-DISCRIMINAÇÃO

O fluxo: analista de subscrição recebe um risco fora de padrão; o copiloto reúne histórico do cliente, sinistralidade da carteira, dados externos e manual de aceitação, e propõe agravo, recusa ou aceitação com ressalva. No caso de retenção, monta a contraproposta dentro de alçada e a submete.

Impacto: reduz tempo de análise e padroniza a aplicação do manual — o ganho de consistência costuma valer mais do que o ganho de velocidade, porque reduz variância de risco na carteira.

Restrição regulatória: critério de aceitação e precificação não pode incorporar variável discriminatória nem *proxy* dela; a recomendação precisa citar a cláusula do manual que a sustenta. Segregação de função: quem propõe não aprova — o agente propõe, o humano com alçada aprova, e a alçada é verificada fora do LLM.

Como o tema do dia muda a decisão: este é o caso que justifica multi-agente de verdade — o sub-agente de dados externos e o sub-agente de manual de aceitação têm permissões genuinamente distintas, e a fronteira de permissão vira a fronteira de agente. É também onde o AgentCore Policy paga por si: alçada e escopo de dado são regras Cedar avaliadas em cada tool call, não instruções em prompt. E é o caso onde a exposição da execution role pelo MMDS mais preocupa: se um documento externo malicioso induzir execução de código, o raio de alcance é exatamente o que a role permite — daí role mínima por sub-agente, container não-root e egress restrito.

7. Anti-padrões

1. Tratar sessão como conexão HTTP. Desenhar o agente como request/response e descobrir em produção que a operação leva 4 minutos, o API Gateway corta antes e o cliente reenvia — gerando efeito colateral duplicado.

Correção: invocação assíncrona com status consultável, streaming para feedback contínuo e chave de idempotência em toda tool com efeito. Timeout do canal e teto de duração do agente precisam ser números conscientes, não herdados do template.

2. Confiar no isolamento da microVM como se fosse o controle de acesso. "Cada sessão tem sua VM, então está isolado." O serviço não impõe o vínculo usuário↔sessão, e qualquer código dentro da VM alcança a execution role pelo MMDS.

Correção: o backend é dono do mapa usuário↔ `sessionId` e do limite de sessões por usuário; `user-id` derivado do principal autenticado, nunca do cliente; execution role escopada como se o conteúdo da VM fosse hostil; container como não-root; e negação explícita de `InvokeAgentRuntimeForUser` onde não for necessária.

3. Prompt como mecanismo de controle. "O sistema instrui o agente a não aprovar acima de R\$ X." Isso é uma preferência, não um controle — e não sobrevive nem a injeção nem a variação estocástica.

Correção: limite verificado fora do modelo — Cedar no Gateway interceptando a tool call, validação de alçada no serviço de destino, e teto de valor no workflow. Guardrails cobrem conteúdo; alçada é autorização.

4. Produção apontando para o endpoint `DEFAULT`. Cada atualização cria versão nova e o `DEFAULT` passa a referenciá-la automaticamente. Funciona perfeitamente até o dia em que um merge chega em produção sem ninguém promover nada.

Correção: endpoints nomeados por ambiente, promoção explícita entre versões imutáveis, prompt e configuração de guardrail versionados junto com a imagem, e rollback = apontar o endpoint para a versão anterior.

5. Observabilidade de aplicação web aplicada a agente. Logar entrada e saída e monitorar latência média. Quando a qualidade cai, ninguém consegue dizer em qual passo — porque o passo não existe como entidade observável.

Correção: trace com span por passo de raciocínio e por tool call, atributos de tenant, versão de agente, versão de prompt, tokens e custo; percentis altos por sessão, não por chamada; e avaliação sobre trajetórias reais, não só sobre a resposta final.

6. Multi-agente como organograma. Criar um agente por time ou por sistema porque “fica mais organizado”. O resultado é latência somada, custo multiplicado por handoff e um trace que ninguém consegue seguir.

Correção: fragmentar por *fronteira de permissão* ou por *dono real*, nunca por estética. Medir na avaliação a degradação de seleção de tool antes de dividir, e exigir que cada handoff justifique a inferência extra que consome.

8. Perguntas de entrevista

► 1. Desenhe a plataforma de agentes que atende quatro LOBs de seguros no mesmo banco. Onde ficam as fronteiras?

► 2. Regulação de sinistro leva de horas a dias e envolve revisão humana. Como você modela isso no runtime?

► 3. Um agente com acesso a tools de apólice foi induzido por um documento enviado pelo cliente a listar dados de outros segurados. O que falhou?

► 4. Trade-off forçado: você tem 6 meses. Runtime gerenciado ou plataforma própria sobre EKS?

► 5. O custo da plataforma triplicou em um mês sem aumento proporcional de volume. Como você investiga?

► 6. Como você garante que a decisão de um agente possa ser auditada seis meses depois?

► 7. Conte de uma vez em que você teve que reduzir a autonomia de um sistema que você mesmo defendeu.

► 8. Cinco times querem construir agentes ao mesmo tempo. Qual é o seu paved road?

9. Números e termos para memorizar

Sessão AgentCore Runtime

1 sessão = 1 microVM dedicada (CPU, memória e filesystem isolados). Duração máxima 8 h; encerramento por inatividade em 15 min; microVM destruída e memória higienizada ao fim.

Versões e endpoints

Versões imutáveis (V1 na criação); endpoints apontam para versões; `DEFAULT` segue a última automaticamente. Estados: `CREATING`, `CREATE_FAILED`, `READY`, `UPDATING`, `UPDATE_FAILED`. Atualização sem downtime.

Inbound × Outbound auth

Inbound: IAM SigV4 ou OAuth 2.0/JWT — um por runtime. Outbound (AgentCore Identity): OAuth e API keys, em modo delegado ao usuário ou autônomo.

Autorização hierárquica

`InvokeAgentRuntime` avalia políticas no runtime e no endpoint — ambos precisam permitir. Deny explícito sempre vence. Cross-account exige resource policy nos dois.

Rede

Subnet privada com NAT (subnet pública não dá internet ao Runtime). PrivateLink:

`com.amazonaws.region.bedrock-agentcore` (data plane) e `...-control` (control plane). Em modo container: endpoints de ECR (dkr e api), S3 gateway e CloudWatch Logs.

Criptografia

Em trânsito: TLS 1.2 no mínimo (recomendado 1.3); WebSocket sobre WSS. Em repouso: chaves AWS owned do KMS por padrão.

runtimeSessionId

Header que dá afinidade à mesma microVM. Ausente ou inconsistente → nova VM, contexto perdido e cold start. Fornecido pelo cliente ou gerado na primeira invocação.

Protocolos suportados

HTTP, streaming, WebSocket bidirecional, MCP (agente↔tool) e A2A (agente↔agente). Processamento assíncrono com status via `/ping`.

MMDS

MicroVM Metadata Service: expõe credenciais da execution role a qualquer código dentro da VM, análogo ao IMDS do EC2. Escopar a role e rodar container como não-root.

Delegação de usuário

`InvokeAgentRuntimeForUser` + header `X-Amzn-Bedrock-AgentCore-Runtime-User-Id`, derivado do principal autenticado. Negar explicitamente onde não for necessário.

Limites de header

Custom headers: 4 KB por valor, 20 headers por runtime. `Authorization` é reservado para agentes com inbound OAuth.

Serviços do AgentCore

Harness, Runtime, Memory, Gateway, Identity, Code Interpreter, Browser, Observability, Evaluations, Optimization, Policy (Cedar), Registry,

Payments. Modulares — usáveis juntos ou isoladamente.

Well-Architected Agentic AI Lens

Publicado em 10/06/2026; seis pilares. Referências úteis: AGENTOPS01/05/06, AGENTREL02/04/06, AGENTSEC03/04/06/07/08, AGENTPERF02/05, AGENTCOST01/02/05, AGENTSUS03.

SRA for AI

OU dedicada de IA generativa com conta de aplicação separada da conta de IA. Riscos nomeados: exfiltração de dados, envenenamento de dados e de template de prompt, exploração de API.

Padrões de orquestração

Agente único · supervisor/arbiter · workflow com nós agentic · swarm. Strands expõe Graph, Swarm e Workflow. Regra: determinismo por padrão, autonomia por exceção.

Contexto regulatório (verificar na fonte)

LGPD: direito de revisão de decisão automatizada. SUSEP: sem norma específica de IA até aqui, atuação por princípios e risco; Plano de Regulação 2026 formalizado por resolução própria — confirme número e vigência antes de citar.

10. Checklist de estudo do dia

- ☐ Reproduzir de memória, em papel, o diagrama da seção 3 — separando plano de controle, plano de dados, fronteira de tenant e observabilidade — em menos de 4 minutos.
- ☐ Escrever o ciclo de vida de uma sessão: criação, afinidade, idle 15 min, teto 8 h, destruição e higienização — e explicar em voz alta por que HITL não pode viver dentro dele.
- ☐ Listar as quatro camadas de fronteira de tenant (compute, credencial, dado, orçamento) e dizer qual serviço AWS implementa cada uma no seu desenho.
- ☐ Ler a página de *security best practices* do AgentCore Runtime e anotar os três controles que você aplicaria primeiro numa seguradora — com a justificativa em uma frase cada.
- ☐ Percorrer o roadmap do Agentic AI Lens (trilha “Moving to production”) e mapear cada best practice ao caso de sinistro da seção 6.
- ☐ Escrever a política de idempotência da plataforma: quem gera a chave, onde ela é armazenada, o que acontece em replay e como a compensação é acionada.

- ☐ Praticar em voz alta a pergunta 4 (trade-off forçado) em até 90 segundos, terminando obrigatoriamente com a condição de reversão.
- ☐ Definir os 5 atributos de span que você exigiria em toda instrumentação de agente — e por que cada um é necessário para responder a uma pergunta de custo ou de auditoria.

FONTES USADAS

- [Amazon Bedrock AgentCore — Runtime: How it works](#) (sessões, microVM, versões, endpoints, protocolos, auth)
- [Amazon Bedrock AgentCore — Use isolated sessions for agents](#)
- [Amazon Bedrock AgentCore — Security best practices for AgentCore Runtime](#) (MMDS, IAM, rede, criptografia, auditoria, responsabilidade compartilhada)
- [Amazon Bedrock AgentCore — Overview](#) (Harness, Gateway, Memory, Identity, Policy, Registry, Evaluations, Optimization)
- [AgentCore Tools — session isolation \(Code Interpreter e Browser\)](#)
- [AWS Well-Architected — Agentic AI Lens](#)
- [AWS Well-Architected Generative AI Lens — Agentic AI](#)
- [AWS Prescriptive Guidance — AWS SRA for AI e Capability: secure access, usage and implementation of generative AI agents](#)
- [AWS Prescriptive Guidance — Security for agentic AI on AWS](#)
- [AWS ML Blog — Building multi-tenant agents with Amazon Bedrock AgentCore](#)

Nota de rigor: números de quota, preço e SLA não foram estimados neste material — onde aparecem limites, eles vêm da documentação citada. Referências regulatórias brasileiras (SUSEP/CNSP, Open Insurance) estão marcadas para confirmação na fonte primária antes de serem citadas em entrevista; os números de mercado da seção 6 estão rotulados como estimativa de dimensionamento.

DIA ANTERIOR — 5

Quando usar modelo comercial, open-weight, fine-tuning, pipelines híbridos ou abordagens não-generativas

PRÓXIMO — DIA 7 (2026-08-07)

Conecta métricas de modelo e de aplicação a outcomes de produto e risco operacional